

Lab 13 · La oficina en una caja

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

60 minutos · Spring Boot 4.1.0 · Java 25 (Temurin)

Resumen

Que la aplicación se empaqueta entera —con su Java dentro— en una imagen de contenedor de **138,9 MB, sin Docker y sin internet**. Y que **el mismo artefacto** se comporta distinto según dónde arranque, sin recompilar nada.

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	3
2. El caso	3
2.1. La mudanza, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · El jar	3
3.2. Paso 2 · Las capas	4
3.3. Paso 3 · Construir la imagen, sin Docker	6
3.4. Paso 4 · Qué hay dentro	7
3.5. Paso 5 · La misma caja, dos comportamientos	8
4. Lo que aprendiste	10
5. Para profundizar	10
6. Antes de cerrar	11

1 Antes de empezar

1.1 Qué vas a lograr

Tu aplicación funciona **en tu máquina**. Hoy la vas a meter en una caja que funciona en cualquier otra.

Vas a empaquetarla en un jar, vas a ver por qué ese jar viene **partido en capas**, vas a construir una **imagen de contenedor sin tener Docker**, vas a abrirla con `tar` para ver qué hay dentro, y vas a comprobar que el mismo artefacto se comporta de dos maneras distintas según una variable de entorno.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs anteriores	Sabes arrancar y configurar	—
Estar en la carpeta del lab	<code>cd labs/lab-13-</code> <code>empaquetado/practica</code>	El <code>cd</code> no da error
NO hace falta Docker	—	Y es el punto del laboratorio

Aquí no se usa `spring-boot:run`. Hoy se construye y se arranca el artefacto: `./mvnw package` y `java -jar`. Es más parecido a lo que pasa en un servidor.

Y para `java -jar` hay que usar el Java del repositorio, no el que tengas instalado. En el paso 1 está el comando y el error que sale si te equivocas.

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. El XML del `pom.xml` lo aguanta —la sangría ahí es adorno—, pero el **YAML no**: los dos archivos de perfil del paso 5 están marcados como «se escribe», y se teclean. El código completo está en `labs/lab-13-empaquetado/solucion/`.

2 El caso

La DGT tiene que llevar su oficina a otro edificio. No basta con llevar los papeles: hay que llevar **los muebles, la instalación eléctrica y el manual de apertura**, y que al llegar todo funcione igual.

2.1 La mudanza, que es la metáfora de este laboratorio

Meter la oficina entera en una caja, y que abra igual en cualquier sitio.

Dentro de la caja va todo lo que la oficina necesita para funcionar: el mobiliario (tu código), las herramientas (las bibliotecas) y **hasta la instalación eléctrica** (el Java). Nadie en el destino tiene que instalar nada ni saber qué versión de nada hace falta.

Y la caja se hace **por capas**, como se embala de verdad:

- Abajo, lo pesado que **nunca cambia**: las estanterías, los archivadores. Son las dependencias.
- Arriba, lo ligero que **cambia cada día**: los papeles de encima de la mesa. Es tu código.

Así, cuando cambias una línea de código, no vuelves a embalar los archivadores: **cambias la caja de arriba**. Kilobytes en vez de ciento treinta y ocho megas.

Y una cosa que NO va dentro de la caja: el cartel de la puerta. El cartel dice si esto es la oficina de pruebas o la de verdad, y a qué Tesorería llama. Eso se pone **al llegar**, en el destino. Si viajara dentro, tendrías que hacer una caja distinta por cada edificio — y entonces la que probaste no sería la que abriste.

3 Los pasos

3.1 Paso 1 · El jar

Qué vamos a hacer

Empaquetar la aplicación en un solo archivo y arrancarla sin Maven.

Para entenderlo mejor

Cerrar la caja. A partir de aquí, lo que se mueve es **un archivo**.

El problema

`./mvnw spring-boot:run` compila, resuelve dependencias y arranca — y para eso necesita Maven, el código fuente y el repositorio de dependencias. En un servidor no hay nada de eso.

La alternativa, y por qué no

- **Un .war desplegado en un Tomcat instalado aparte:** es como se hacía, y significa que tu aplicación **no se puede arrancar sola**. Depende de una máquina que preparó otra persona, y lo que pruebas no es lo que corre.
- **Un jar ejecutable con el servidor dentro,** que es lo de aquí: `java -jar` y ya está. Lo mismo en tu portátil y en producción.

Se corre

```
./mvnw package
```

Y para arrancarlo, **con el Java del repositorio:**

```
../../../../tools/jdk/runtime/macos-aarch64/jdk-25.0.4+7/Contents/Home/bin/java \
-jar target/lab13-empaquetado-0.1.0.jar
```

Lo que vas a ver

```
target/lab13-empaquetado-0.1.0.jar    20.9 MB
```

Veinte megas: tu código es una parte mínima; el resto son Spring, Tomcat y las bibliotecas.

Vas bien si...

El package termina en `BUILD SUCCESS` y en `target/` hay un `.jar` de unos 20 MB.

Si te atascas

1 · `UnsupportedClassVersionError ... class file version 69.0 ... up to 65.0`

Exception in thread "main" java.lang.UnsupportedClassVersionError:
cl/dgt/empaquetado/Lab13Application has been compiled by a more recent version of
↪ the
Java Runtime (class file version 69.0), this version of the Java Runtime only
recognizes class file versions up to 65.0

Estás arrancando con el Java de tu máquina, no con el del repositorio. El jar está compilado con Java 25 (versión de clase 69) y tu java es más viejo — 65 es Java 21.

Comprueba cuál tienes con `java -version`, y usa la ruta completa al Java del repositorio, como en el comando de arriba. `./mvnw` lo hace solo; `java -jar` no.

2 · `no main manifest attribute`

Empaquetaste sin el plugin de Spring Boot, así que el jar no sabe arrancarse. Comprueba que el `spring-boot-maven-plugin` está en el `pom.xml`.

3.2 Paso 2 · Las capas

Qué vamos a hacer

Partir el jar por velocidad de cambio, y mirar las cuatro capas.

Para entenderlo mejor

Embalar por separado lo que nunca cambia y lo que cambia a diario.

El problema

Un jar es **una sola pieza**. Si cambias una línea de código, la pieza entera es distinta — y en una imagen de contenedor eso significa volver a subir y bajar los 138 megas enteros por una letra.

La alternativa, y por qué no

- **Jar plano**: lo de fábrica, y perfecto cuando sólo copias el jar a un servidor. Ahí las capas no aportan nada.
- **Jar por capas**, que es lo de aquí: sólo tiene sentido cuando hay una imagen de por medio — y entonces cambia mucho, porque lo pesado deja de viajar.

Se pega

En practica/pom.xml, **dentro del plugin de Spring Boot**:

```
<configuration>
  <layers>
    <enabled>true</enabled>
  </layers>
</configuration>
```

Se corre

```
./mvnw package
<el java del repositorio> -Djarmode=tools -jar target/lab13-empaquetado-0.1.0.jar
↪ list-layers
```

Lo que vas a ver

```
dependencies
spring-boot-loader
snapshot-dependencies
application
```

Cuatro capas, ordenadas de menos a más volátil. dependencies es casi todo el peso y cambia cuando cambias una versión —una vez cada varios meses—. application es tu código y cambia cada vez que guardas.

Vas bien si...

El comando lista las cuatro capas, con application la última.

Si te atascas

1 · Unknown jarmode

No activaste las capas en el pom.xml, o no volviste a empaquetar después de activarlas.

3.3 Paso 3 · Construir la imagen, sin Docker

Qué vamos a hacer

Empaquetar la aplicación en una imagen de contenedor **sin tener Docker instalado**.

Para entenderlo mejor

Cerrar la caja del todo: con la instalación eléctrica —el Java— dentro.

El problema

Construir una imagen normalmente exige **un demonio de Docker corriendo**, y las máquinas del SII no tienen Docker ni pueden instalarlo.

La alternativa, y por qué no

- **Un Dockerfile con docker build**: lo que hace todo el mundo, y **exige el demonio**.
- **spring-boot:build-image** (Cloud Native Buildpacks): también exige el demonio.
- **Jib**, que es lo de aquí: construye la imagen **desde Maven**, sin demonio y sin escribir un Dockerfile. Lee el proyecto, arma las capas y escribe el resultado.

Su límite, y conviene saberlo: **Jib no ejecuta comandos dentro de la imagen**, sólo copia archivos. Si necesitaras instalar un paquete del sistema, harían falta las otras opciones.

Y la imagen base viaja en el repositorio. El `.mvn/maven.config` de este proyecto lleva `-Djib.baseImageCache=../../tools/jib-base`, y ahí están las capas de `eclipse-temurin:25-jre` commiteadas. Sin eso, Jib intentaría bajarla de un registro — y aquí no hay internet.

Se pega

En `practica/pom.xml`, **dentro de `<plugins>`**. La versión sale de la propiedad `jib.version`, que el `pom.xml` ya trae declarada arriba:

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
  <version>${jib.version}</version>
  <configuration>
    <from>
      <image>eclipse-temurin:25-jre</image>
    </from>
    <to>
      <image>lab13-empaquetado:0.1.0</image>
    </to>
    <container>
      <ports>
        <port>8106</port>
      </ports>
      <environment>
        <SPRING_PROFILES_ACTIVE>prod</SPRING_PROFILES_ACTIVE>
      </environment>
    </container>
  </configuration>
</plugin>
```

```
</configuration>
</plugin>
```

Se corre

```
./mvnw package jib:buildTar
```

Lo que vas a ver

```
[INFO] Containerizing application to file at 'target/jib-image.tar'...
[INFO] Built image tarball at target/jib-image.tar
[INFO] BUILD SUCCESS
```

```
target/jib-image.tar 138.9 MB
```

Una imagen de contenedor, sin Docker y sin internet.

Vas bien si...

Existe target/jib-image.tar y pesa alrededor de 138 MB.

Si te atascas

1 · Cannot run Jib in offline mode; eclipse-temurin:25-jre not found in local Jib cache

Falta la caché de la imagen base, o el .mvn/maven.config no la apunta. Comprueba que existe tools/jib-base/ en el repositorio.

2 · Jib intenta salir a la red y falla.

Lo mismo: sin la caché local, va al registro.

3.4 Paso 4 · Qué hay dentro

Qué vamos a hacer

Abrir la imagen con tar y mirar. Una imagen **no es magia**: es un tar con capas y un JSON.

Para entenderlo mejor

Abrir la caja en el destino y comprobar el inventario antes de montar nada.

Se corre

```
mkdir -p /tmp/img && tar -xf target/jib-image.tar -C /tmp/img
ls /tmp/img
```

Lo que vas a ver

Un puñado de `.tar.gz` —las capas—, un `manifest.json` y un JSON de configuración. Y dentro de la configuración:

```
capas en la imagen: 10
entrypoint: java -cp @/app/jib-classpath-file cl.dgt.empaquetado.Lab13Application
puerto: ['8106/tcp']
env SPRING_PROFILES_ACTIVE: ['SPRING_PROFILES_ACTIVE=prod']
```

Tres cosas que vale la pena mirar:

- El **entrypoint** es un `java -cp` normal y corriente. Nada exótico.
- El **puerto** que declaraste en el `pom.xml`.
- **SPRING_PROFILES_ACTIVE=prod**, grabado dentro. Un contenedor arrancado sin decir nada se comporta como producción — que es el valor por defecto **seguro**: al revés, un despliegue mal hecho apuntaría a pruebas sin quejarse.

Vas bien si...

Puedes listar las capas y leer el `entrypoint` y el puerto en el JSON de configuración.

3.5 Paso 5 · La misma caja, dos comportamientos

Qué vamos a hacer

Arrancar **el mismo jar** tres veces y obtener tres comportamientos, sin recompilar nada.

Para entenderlo mejor

El cartel de la puerta. La caja es la misma; lo que cambia es lo que pones al llegar.

El problema

Si la configuración viaja dentro del artefacto, hace falta **un artefacto por entorno**. Y entonces el que probaste no es el que desplegaste — que es exactamente la clase de diferencia que produce los incidentes que nadie sabe reproducir.

La alternativa, y por qué no

- **Una imagen por entorno**: se hace, y sólo tiene sentido cuando cambian las dependencias y no la configuración.
- **Perfiles + variables de entorno**, que es lo de aquí: un artefacto, y el entorno decide. Y hay un orden de precedencia: **la variable de entorno gana al perfil, y el perfil gana al `application.yml`**.

Se escribe — aquí no se pega nada

El YAML se escribe a mano: la sangría es el significado. Al copiarlo del PDF se pierden los espacios del principio de línea y el archivo deja de decir esto, **sin dar error**. Son **dos espacios por nivel**, y ninguna tabulación.

Archivo **nuevo** `practica/src/main/resources/application-dev.yml` — el archivo entero:

lab12:

```
saludo: Hola desde DESARROLLO
tesoreria-url: http://localhost:9098/pagos
```

Y practica/src/main/resources/application-prod.yml — entero:

lab12:

```
saludo: Hola desde PRODUCCIÓN
tesoreria-url: ${TESORERIA_URL:https://tesoreria.example.cl/pagos}
```

Lee despacio esa última línea, que es la que enseña la precedencia:

```
${NOMBRE_DE_VARIABLE:valor-de-respaldo}
```

Si existe la variable de entorno, gana ella. Si no, el texto de después de los dos puntos.

Y ese respaldo es un respaldo, no el valor de producción. `example.cl` no existe **a propósito**: si algún día esa URL aparece en un log de producción, es que nadie puso la variable — y se ve enseguida, en vez de apuntar en silencio a un sitio equivocado.

Se corre

```
SPRING_PROFILES_ACTIVE=dev <java> -jar target/lab13-empaquetado-0.1.0.jar
SPRING_PROFILES_ACTIVE=prod <java> -jar target/lab13-empaquetado-0.1.0.jar
SPRING_PROFILES_ACTIVE=prod TESORERIA_URL=https://tesoreria.sii.cl/api/pagos \
    <java> -jar target/lab13-empaquetado-0.1.0.jar
```

y en cada caso: `curl localhost:8105/donde-estoy`

Lo que vas a ver

```
dev:
{"saludo":"Hola desde DESARROLLO","tesoreriaUrl":"http://localhost:9098/pagos",
 "perfilesActivos":["dev"],"javaVersion":"25.0.4"}

prod, sin variable:
{"saludo":"Hola desde
↪ PRODUCCIÓN","tesoreriaUrl":"https://tesoreria.example.cl/pagos",
 "perfilesActivos":["prod"],"javaVersion":"25.0.4"}

prod, con TESORERIA_URL:
{"saludo":"Hola desde
↪ PRODUCCIÓN","tesoreriaUrl":"https://tesoreria.sii.cl/api/pagos",
 "perfilesActivos":["prod"],"javaVersion":"25.0.4"}
```

Mismo jar. Tres comportamientos. Y fíjate en el `javaVersion`: **25.0.4 en los tres**, que es el Java que viaja en la caja.

Y la regla que cierra el laboratorio: NINGÚN SECRETO en estos archivos.

Ni la contraseña de la base, ni el secreto de firma del JWT del Lab 09, ni una clave de API. Y la razón es concreta, no una buena práctica abstracta: **estos archivos están DENTRO de**

la imagen, y una imagen se copia, se comparte, se sube a un registro y **se abre con tar** — como acabas de hacer en el paso 4. Cualquiera que tenga la imagen tiene estos archivos.

Además están en el repositorio, así que el secreto quedaría en el historial de Git **para siempre**, y borrarlo después no lo borra.

Los secretos van por **variable de entorno**, que las pone el orquestador al arrancar y no quedan escritas en ninguna parte.

Vas bien si...

Los tres arranques dan tres respuestas distintas, y en los tres el `javaVersion` es 25.0.4.

Si te atascas

1 · Los tres dicen lo mismo.

El perfil no está llegando. En Windows, `SPRING_PROFILES_ACTIVE=prod java -jar ...` **no funciona**: hay que usar `set` antes, o pasarlo como `--spring.profiles.active=prod`.

2 · Could not resolve placeholder 'TESORERIA_URL'

Escribiste `${TESORERIA_URL}` sin respaldo en un perfil que no lo define. Es el comportamiento correcto en producción para un secreto — pero para esta URL hay respaldo.

4 Lo que aprendiste

1 · Un jar ejecutable lleva el servidor dentro.

`java -jar` y ya está: lo mismo en tu portátil y en el servidor. No hay que instalar ni configurar un contenedor de aplicaciones aparte.

2 · Las capas existen para no mover lo que no cambió.

Dependencias abajo, tu código arriba. Cambiar una línea mueve kilobytes en vez de 138 MB — y sólo importa cuando hay una imagen de por medio.

3 · Se puede construir una imagen sin Docker.

Jib lo hace desde Maven, con la imagen base viajando en el repositorio. Y la imagen no es magia: es un tar con capas y un JSON que puedes abrir y leer.

4 · El mismo artefacto, y el entorno decide.

Perfiles y variables de entorno, con una precedencia clara. Un artefacto por entorno rompe la única garantía que tienes: que lo que probaste es lo que desplegaste. Y ahí dentro **no van secretos**.

5 Para profundizar

- **Cambia una línea de tu código**, vuelve a construir la imagen, y mira cuáles de las capas cambian de huella.
- **Quita** `<layers>` y compara el resultado.
- **Cambia la imagen base** a `eclipse-temurin:25-jdk` y compara el tamaño. ¿Cuánto cuesta llevar el compilador que no vas a usar?
- **Arranca sin** `SPRING_PROFILES_ACTIVE` y mira qué perfil sale. ¿Por qué es más seguro que salga ése?

- **Busca distroless** y piensa qué ganarías y qué perderías si no pudieras entrar a la imagen a mirar.

6 Antes de cerrar

Este lab no deja procesos vivos si paraste los `java -jar` con `Ctrl+C`. Si alguno quedó suelto:

```
lsof -ti:8105 | xargs kill -9
./mvnw clean
```

Lo que te llevas:

La aplicación viaja entera — con su Java — en una imagen por capas que se construye sin Docker. La configuración se pone al llegar, no dentro. Y dentro de la caja no van secretos, porque la caja se abre con `tar`.

Lo que queda pendiente, y abre el Lab 14: todo esto es **una** aplicación. En el Lab 14 la DGT se parte en cuatro procesos con tres bases distintas, y aparecen los problemas que sólo existen cuando el trabajo está repartido.